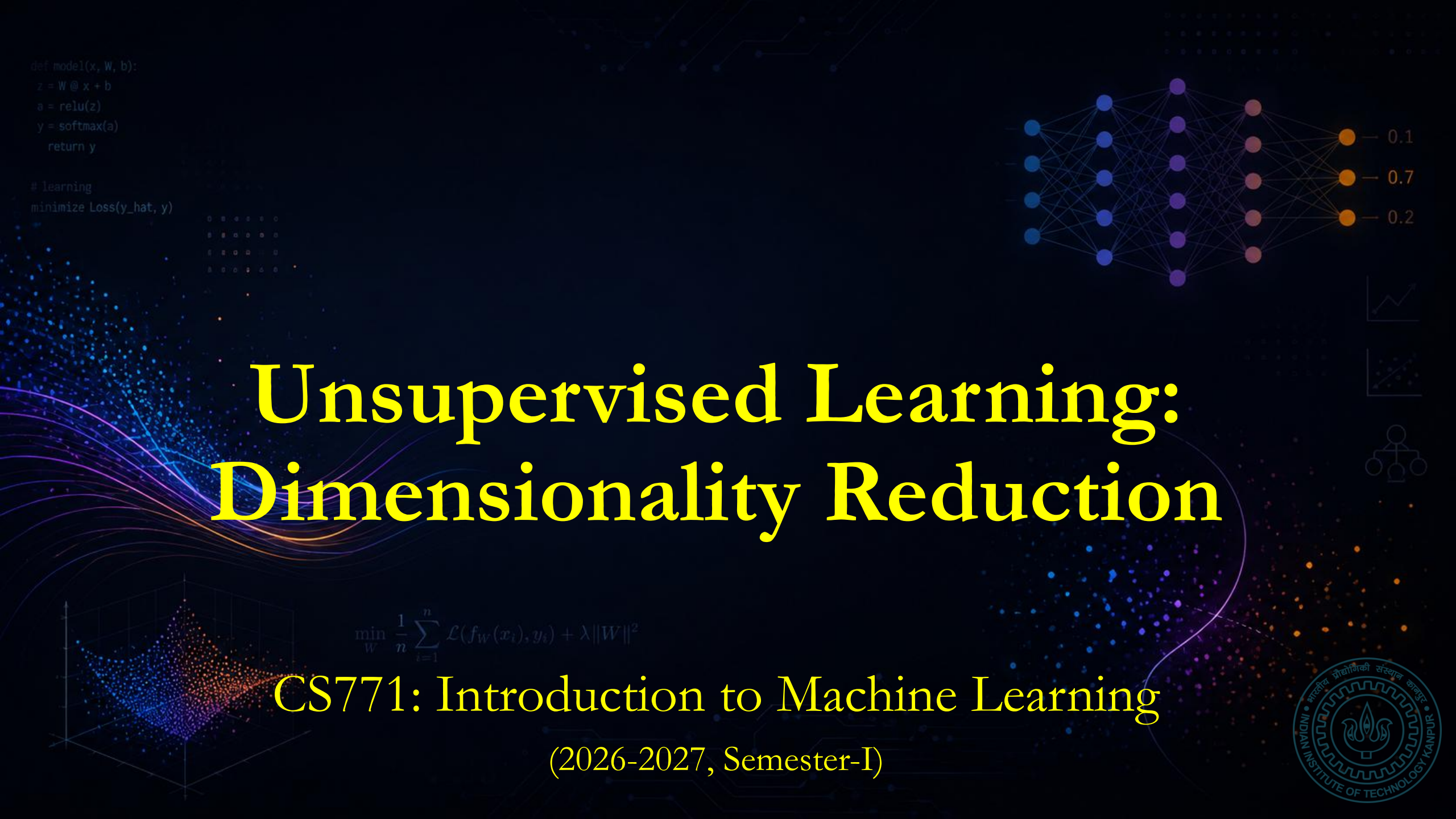


Unsupervised Learning: Dimensionality Reduction

CS771: Introduction to Machine Learning

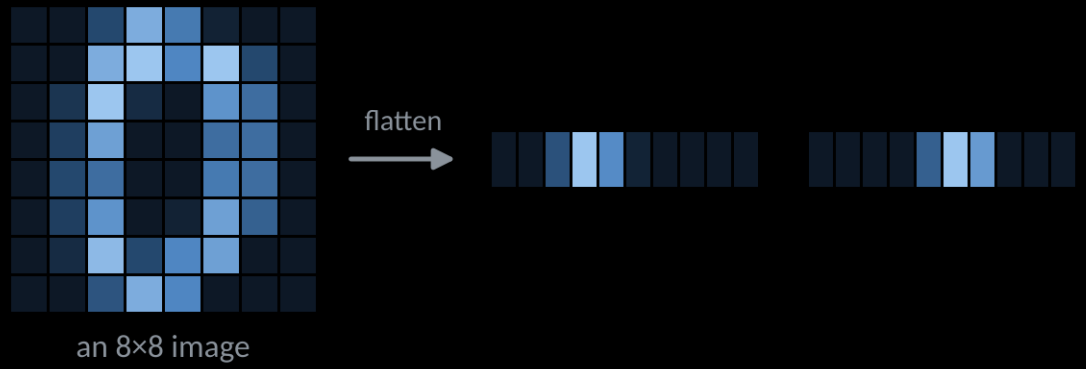
(2026-2027, Semester-I)

Dimensionality Reduction

In ML, high-dimensionality can arise in two senses

Data: $\boldsymbol{x}_n \in \mathbb{R}^D$

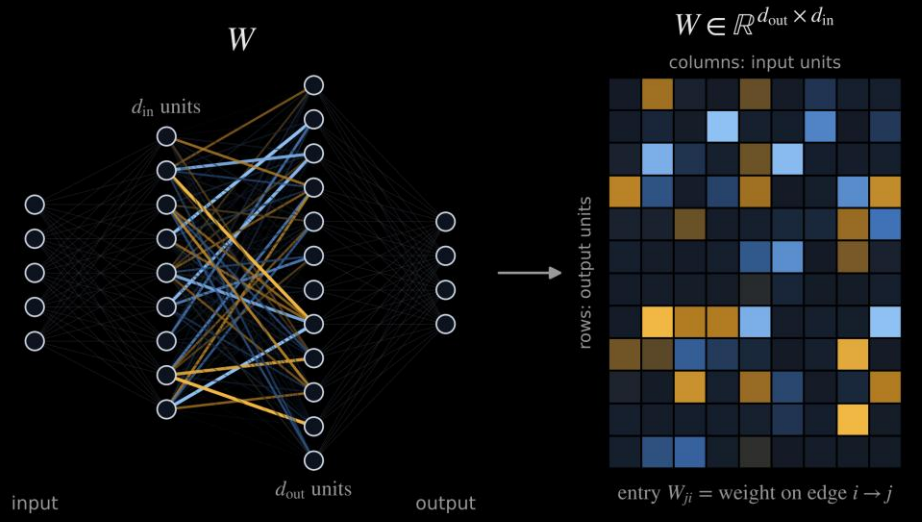
Data may be in form of vector, matrix, tensor, etc, and D is the size after flattening



- A gene-expression sample may contain 10,000 measured features.
- An image, document representation, or embedding may live in thousands or millions of dimensions

Parameters/weights: $\boldsymbol{W} \in \mathbb{R}^{d_{out} \times d_{in}}$

May be in form of vector, matrix, tensor, etc, and $P = d_{out} \times d_{in}$ is size after flattening



- A neural network may contain millions or billions of weights (high weight dimensionality)

Often, low-dimensional structures in data and parameters suffice

Dimensionality reduction is about identifying such low-dim structures

Dimensionality Reduction

High-dimensional data

Each observation has many measured coordinates.

Example: $\boldsymbol{x_n} \in \mathbb{R}^{10{,}000}$

Goal: find compact coordinates $z_n \in \mathbb{R}^K$

Assume a function $f: \mathbb{R}^D \rightarrow \mathbb{R}^K$



many measured coordinates → a few compact coordinates

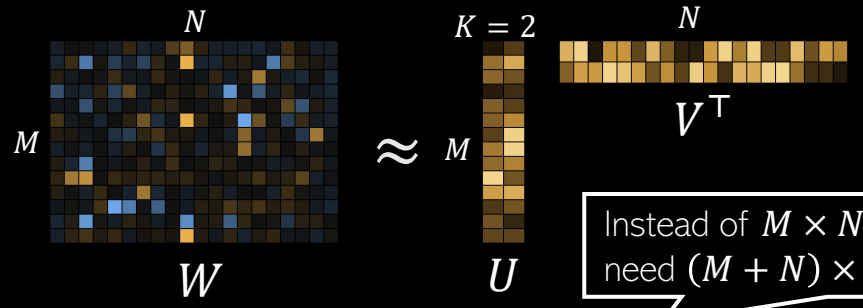
$$x_n \in \mathbb{R}^D \longrightarrow z_n = f(x_n) \in \mathbb{R}^K, \quad K \ll D$$

High-dimensional parameters

A model has many learnable weights.

Example: $W \in \mathbb{R}^{\text{one million}}$

Goal: Approximate massive number of weights using a compact set of weights



Popular especially for LLMs, deep learning

Learn U, V instead of learning W

Instead of $M \times N$ parameters, we need $(M + N) \times K$ parameters only

Weight matrix assumed to be approximately "low-rank"

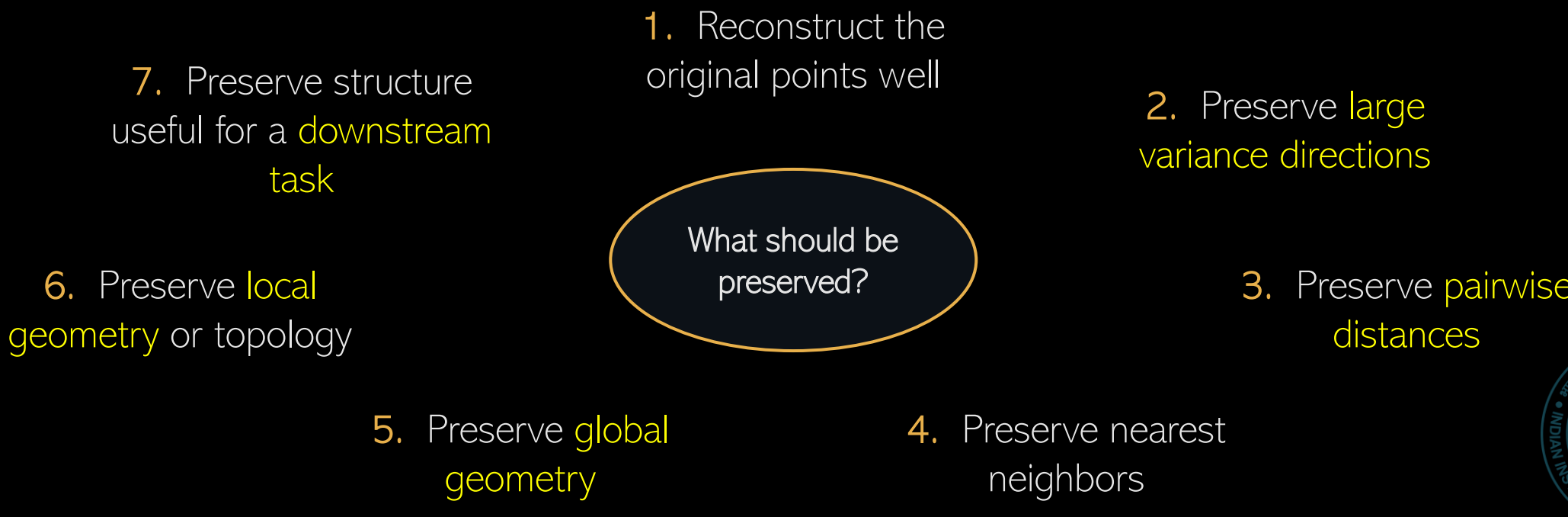
$$W \approx UV^{\mathsf{T}}$$

Assumption: intrinsic dimension << extrinsic dimension in the above cases

What Should Be Preserved When Doing Dim-Red?

Dimensionality reduction does compression and can potentially throw away information in the process

Different dimensionality reduction methods differ primarily in what they choose to preserve

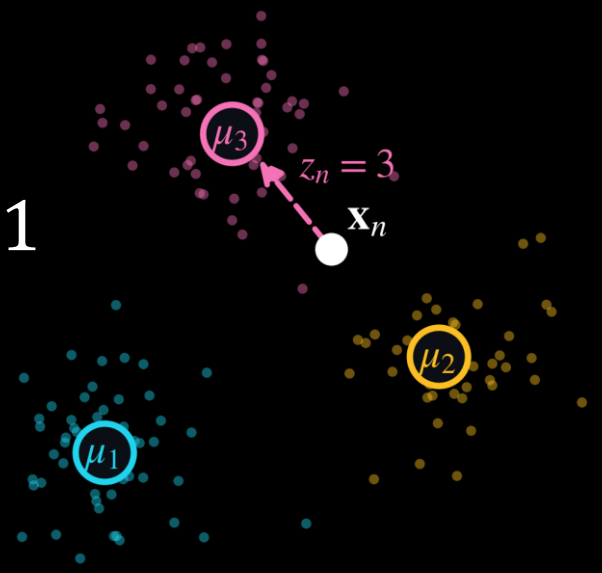


K-means Revisited: As a Dim. Reduction Method

K-means quantized $\boldsymbol{x}_n \in \mathbb{R}^D$ by its closest cluster mean: $\boldsymbol{x}_n \approx \boldsymbol{\mu}_{z_n}$

Using a one-hot encoding for $z_n \in \{1,2,\dots,K\}$

$$\boldsymbol{x}_n \approx \sum_{k=1}^{K} z_{nk}\boldsymbol{\mu}_k \qquad z_{nk} \in \{0,1\}, \qquad \sum_{k=1}^{K} z_{nk} = 1$$



K-means as a compression method:

- Run K-means on data $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \dots, \boldsymbol{x}_N\}$
- Store only $\boldsymbol{Z} = \{\boldsymbol{z}_1, \boldsymbol{z}_2, \dots, \boldsymbol{z}_N\}$ and means $\boldsymbol{\mu} = \{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_K\}$

  Storage saving: $\boxed{ND \rightarrow NK + DK = (N + D)K}$

- Reconstruction of $\boldsymbol{x}_n$ is $\hat{\boldsymbol{x}}_n = \boldsymbol{\mu}_{z_n}$. Loss $= \|\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n\|$

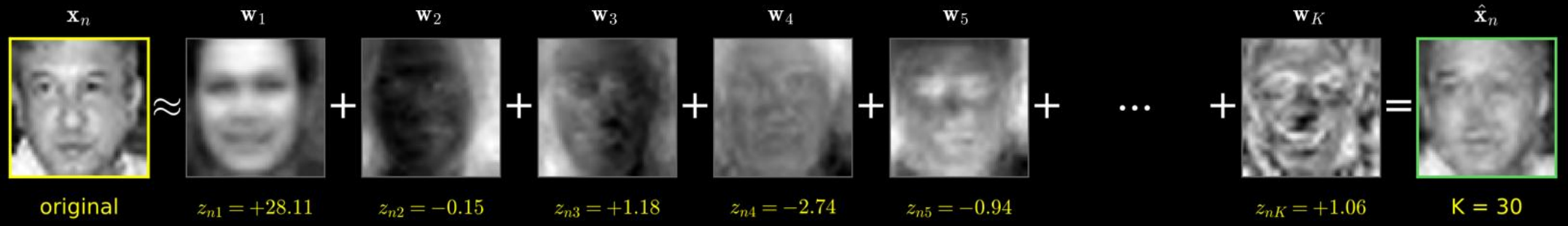
A Better Quantization

Assumed to be columns of a $D \times K$ matrix $\boldsymbol{W}$

Let's approximate $\boldsymbol{x}_n$ as weighed sum of D-dim vecs $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_K$

$$\boldsymbol{x}_n \approx \sum_{k=1}^{K} z_{nk} \boldsymbol{w}_k = \boldsymbol{W} \boldsymbol{z}_n$$

In the above, $\boldsymbol{z}_n \in \mathbb{R}^K$ is not a one-hot vector unlike K-means case



The image has dimensionality $D = 64 \times 64 = 4096$

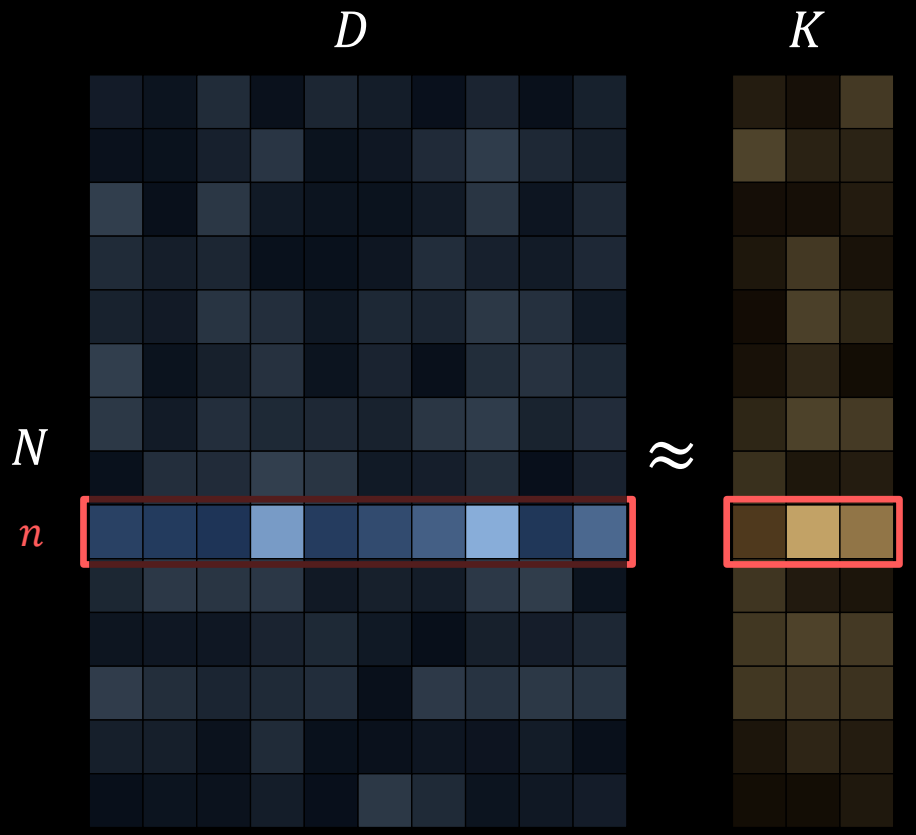
An object expressed as a weighted combination of a set of "basis" objects

The 4096-dim image is approximated using K numbers

Quantization gives an encoding/embedding $\boldsymbol{z}_n$ for each $\boldsymbol{x}_n$

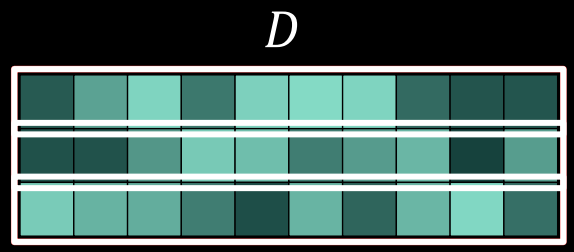
Matrix Factorization View

$\boldsymbol{X}$ approximated by a product of two thin matrices $\boldsymbol{Z}$ and $\boldsymbol{W}$



$\boldsymbol{X} \in \mathbb{R}^{N \times D}$

$\boldsymbol{Z} \in \mathbb{R}^{N \times K}$



$\boldsymbol{W}^\top \in \mathbb{R}^{K \times D}$

$$\boldsymbol{X} \approx \boldsymbol{Z}\boldsymbol{W}^\top$$

$$\boldsymbol{x}_n^\top \approx \boldsymbol{z}_n^\top \boldsymbol{W}^\top$$

$$\{\widehat{\boldsymbol{Z}}, \widehat{\boldsymbol{W}}\} = \operatorname{argmin}_{\boldsymbol{Z},\boldsymbol{W}} \|\boldsymbol{X} - \boldsymbol{Z}\boldsymbol{W}^\top\|^2$$

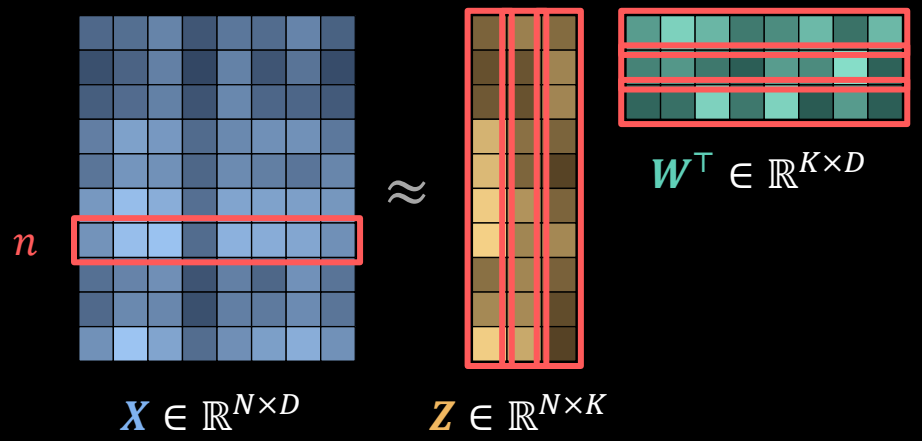
Thus matrix factorization of $\boldsymbol{X}$ can be used for dimensionality reduction

Can be solved using GD/SGD or ALT-OPT

Various types of constraints can also be imposed on $\boldsymbol{Z}$ and $\boldsymbol{W}$ (e.g., non-negativity, sparsity, etc)

Thus many variants exist for matrix factorization

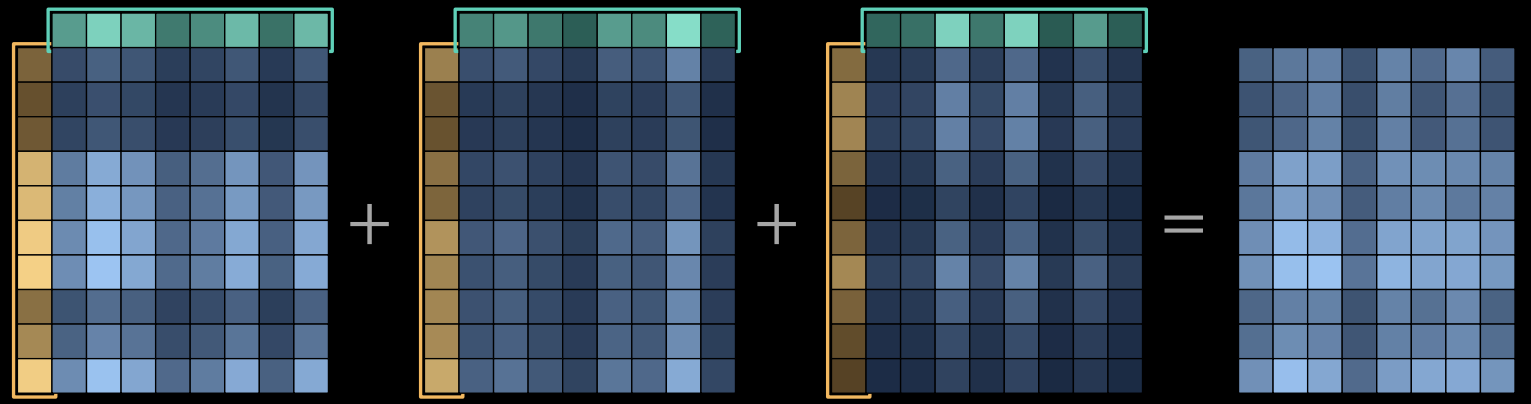
Matrix Factorization as Sum of Rank-1 Matrices



$$X \approx ZW^\top = \sum_{k=1}^{K} z_{:k} w_k^\top$$

$z_{:k}$ = column k of Z

w_k = column k of W = row k of $W^\top$



A sum of K rank-1 matrices $\Rightarrow \text{rank}(ZW^\top) \leq K$

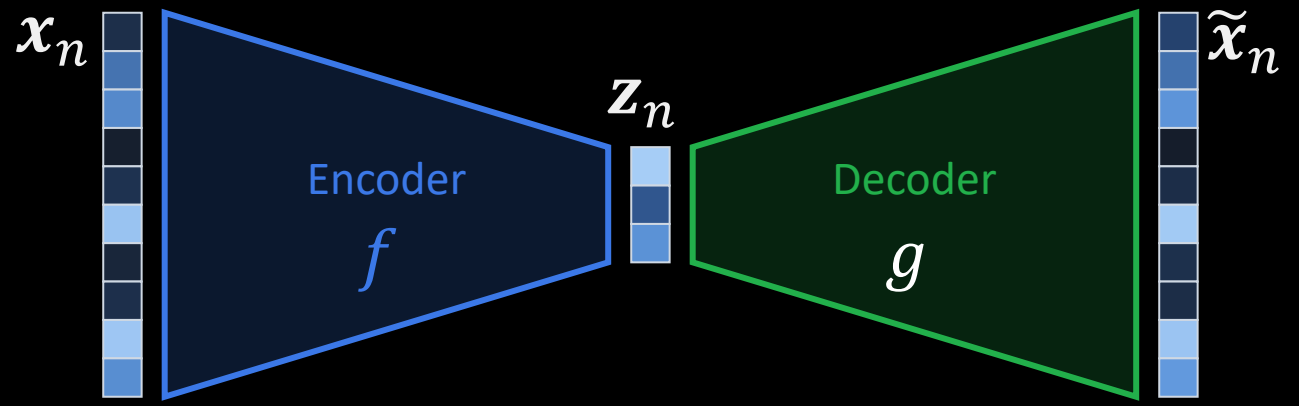
Thus X approximated by an at most rank K matrix (and if $K < \min\{N, D\}$, we say that it is a "low rank approximation" of the matrix X)

Dim-Red: A General Encoder-Decoder View

The encoder-decoder view

$$\boldsymbol{z}_n = f(\boldsymbol{x}_n)$$ (Encoder)

$$\tilde{\boldsymbol{x}}_n = g(\boldsymbol{z}_n) = g(f(\boldsymbol{x}_n)) \approx \boldsymbol{x}_n$$ (Decoder)



Usually f and g are learned by minimizing total reconstruction error

Reconstruction error of one input

$$\|\boldsymbol{x}_n - g(f(\boldsymbol{x}_n))\|^2$$

$$\{\hat{f}, \hat{g}\} = \arg\min_{f,g} \mathcal{L}(f, g)$$

$$= \arg\min_{f,g} \sum_{n=1}^{N} \|\boldsymbol{x}_n - g(f(\boldsymbol{x}_n))\|^2$$

Various forms possible for f and g, Can be linear/nonlinear functions

A Linear Encoder and A Linear Decoder

Consider a linear model with a $D \times K$ weight matrix $\boldsymbol{W}$ as encoder

$$\boldsymbol{z}_n = f(\boldsymbol{x}_n) = \boldsymbol{W}^\top \boldsymbol{x}_n \in \mathbb{R}^K$$

Suppose decoder is also linear with transposed weights of encoder

$$\tilde{\boldsymbol{x}}_n = g(\boldsymbol{z}_n) = \boldsymbol{W}\boldsymbol{z}_n = \sum_{k=1}^{K} z_{nk}\boldsymbol{w}_k \in \mathbb{R}^D$$

Then the total reconstruction loss for $N \times D$ data matrix $\boldsymbol{X}$ is

$$\mathcal{L}(W) = \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{W}\boldsymbol{z}_n\|^2 = \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{W}\boldsymbol{W}^T\boldsymbol{x}_n\|^2$$

Result: If the K columns of $\boldsymbol{W}$ are mutually orthogonal then

Top K eigenvectors of the covariance matrix of $\boldsymbol{X}$

$$\boldsymbol{W} = \mathbf{eigvec}[\mathbf{cov}(X), K]$$

"Principal Component Analysis" (PCA)

Identifying Directions for Projection



3-D object



Light from the side

coils and height survive



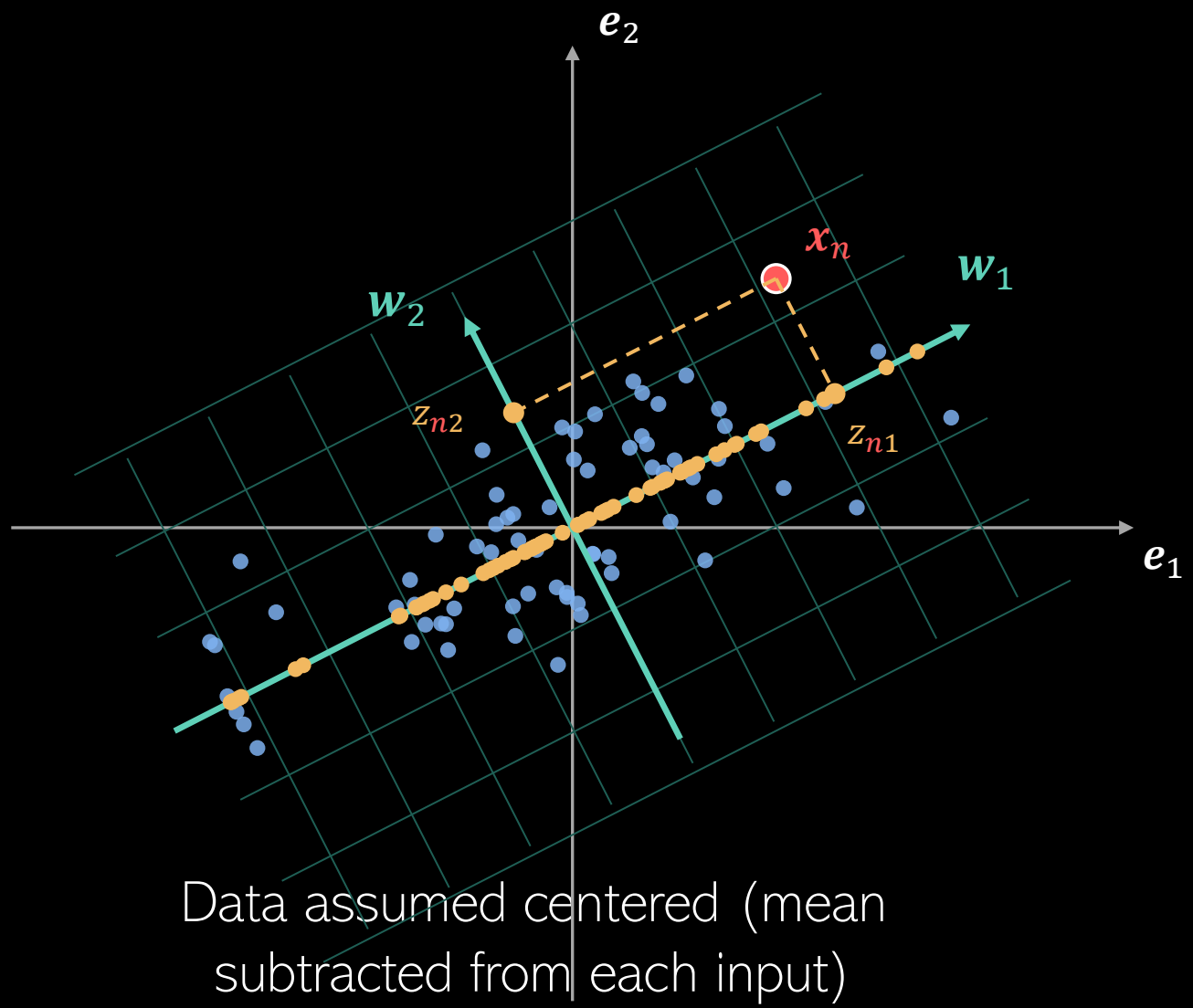
Light from above

collapses to a circle — height is lost

How do we choose the “best” angle for the light?

We need a mathematical proxy for “information”.

Principal Component Analysis (PCA)



Data assumed centered (mean subtracted from each input)

In the standard axes e_1, e_2, both coordinates vary a lot: dropping either one loses information.

PCA rotates to new orthonormal axes aligned with the data with w_1 = largest spread, w_2 = smallest spread directions

$$x_n = z_{n1} w_1 + z_{n2} w_2$$

new coordinates = projections: $z_{nk} = w_k^\top x_n$

Let's drop direction w_2 (since very little spread):

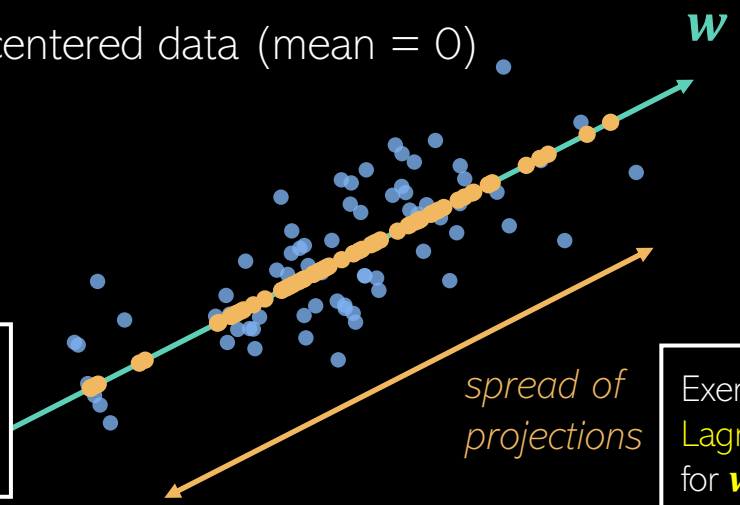
$$x_n \approx \hat{x}_n = z_{n1} w_1$$

2 numbers → 1 number per point. In general keep $K \ll D$ directions as the columns of $W \in \mathbb{R}^{D \times K}$:

$$z_n = W^\top x_n, \qquad \hat{x}_n = W z_n$$

Principal Component Analysis (PCA)

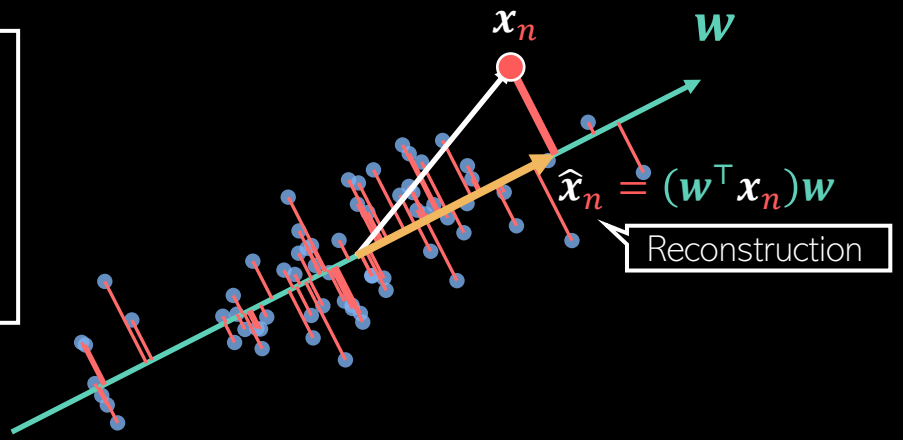
Assume centered data (mean = 0)



Mean of the projections is zero since data is centered already

This defines the largest variance direction, call it w_1. Next best direction w_2 is orthogonal to w_1, and successive directions $w_3, w_4, \ldots, w_K$ defined likewise (orthogonal to all previous ones)

Exercise: Construct the Lagrangian and solve for w to verify that w is the largest eigenvector



Maximize the variance of the projections

$$\max_{\|w\|=1} \frac{1}{N}\sum_{n=1}^{N}(w^\top x_n)^2 = \max_{\|w\|=1} w^\top S w$$

Minimize the reconstruction error

$$\min_{\|w\|=1} \sum_{n=1}^{N}\|x_n - (w^\top x_n)w\|^2 = \min_{\|w\|=1} \sum_{n=1}^{N}\|x_n - w(w^\top x_n)\|^2$$

$$\underbrace{\|x_n\|^2}_{\text{fixed}} = \underbrace{(w^\top x_n)^2}_{\text{captured}} + \underbrace{\|x_n - \hat{x}_n\|^2}_{\text{error}}$$

Pythagoras theorem, for every point. The total (norm of x_n) is fixed, so more variance captured ⇔ less rec. error.

Both views ⇒ $w_1, \ldots, w_K$ = top-K eigenvectors of $S = \frac{1}{N}X^\top X$, variance captured along $w_k = \lambda_k$ (k-th eigval)

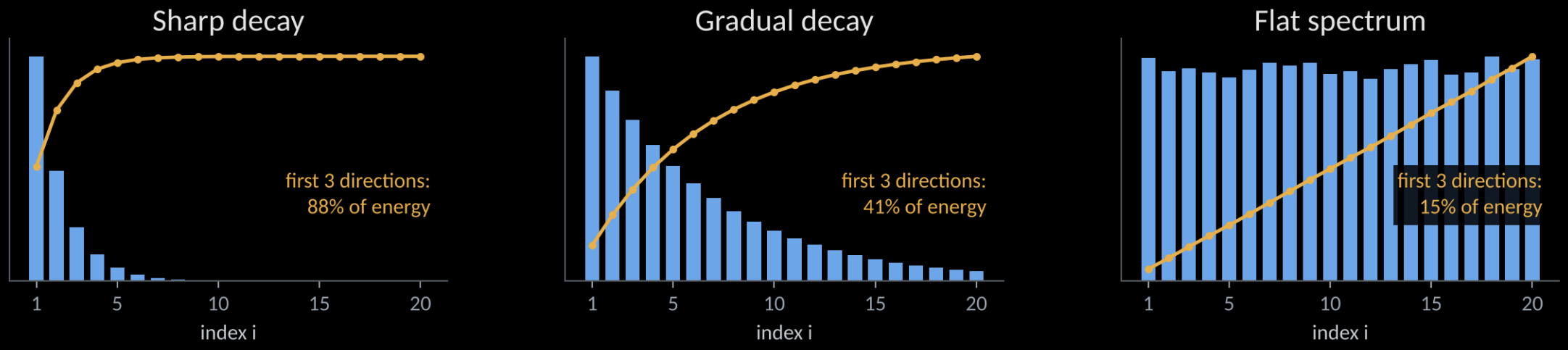
Eigenvalues tell us how much dim-red is possible

$$\text{Eigenvalue spectrum} = \{\lambda_1, \lambda_2, \dots, \lambda_D\}$$

bars: eigenvalues λ_i (scaled)

$$\text{cumulative energy} = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{D} \lambda_i}$$

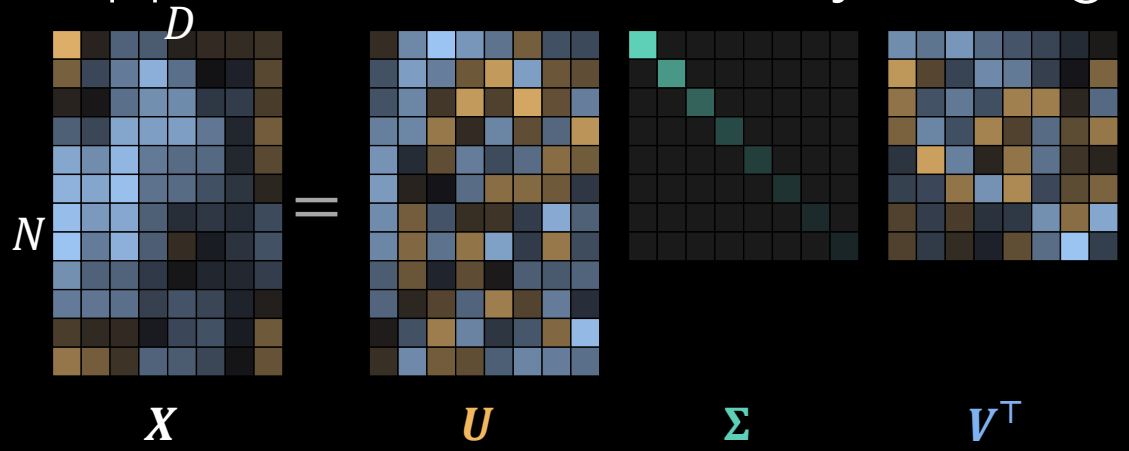
line: cumulative energy



- Sharp decay: a few directions dominate.
- Gradual decay: moderate compression may be possible.
- Flat spectrum: no obvious small linear dim-red possible

Singular Value Decomposition (SVD)

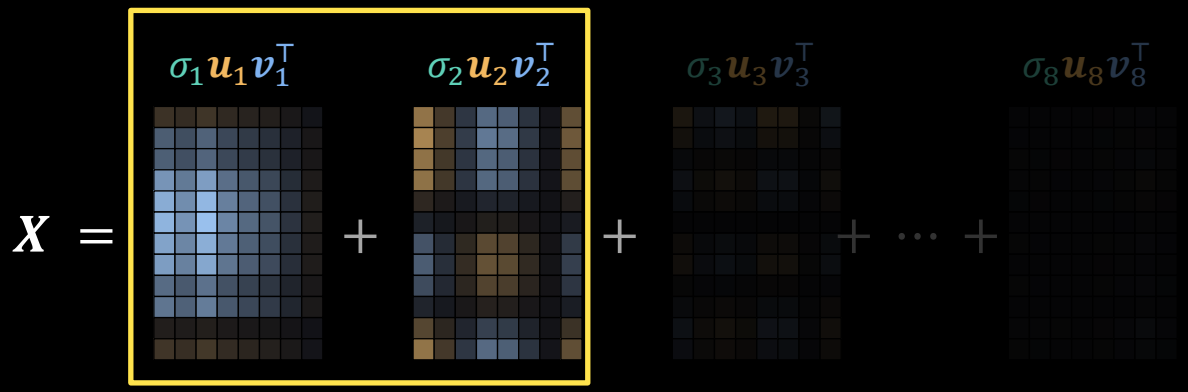
Approximate a matrix by a weighted sum of K rank-1 pieces



$$X = U\Sigma V^\top = \sum_{k=1}^{r=\min\{N,D\}} \sigma_k u_k v_k^\top$$

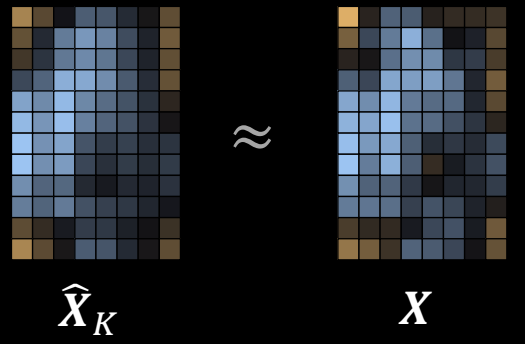
U, V: orthonormal columns (left / right singular vectors)

$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$: singular values (diagonal of Σ)



later pieces carry less and less info about X

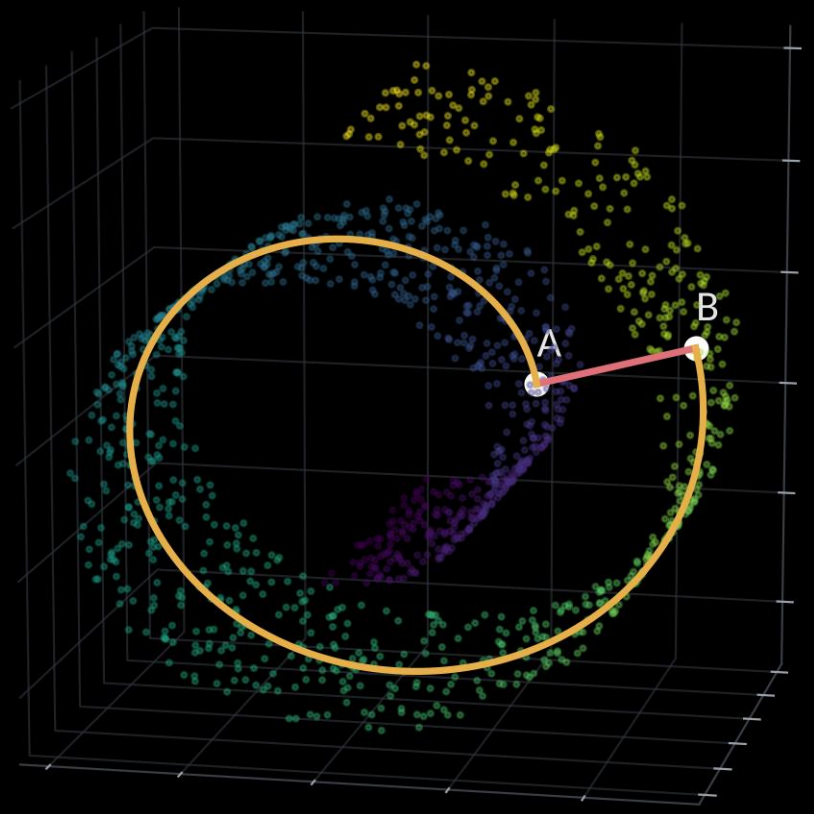
Keep the top $K = 2$ pieces:



Best rank-K approximation: smallest $\|X - \hat{X}_K\|$ *(Eckart–Young)*

Link to PCA (centered X): V_K (first K columns of V) $= W$ (in PCA) $\quad Z = XW = U_K\Sigma_K \quad \lambda_k = \frac{\sigma_k{}^2}{N}$

When linear projections don’t work



Instead of preserving variance or reconstruction error, in some cases, preserving local geometry may be more appropriate

Focusing on preserving the local neighborhood may be a better idea